OakGPT Phase 3.3: Timing Attribution and Development Reliability Audit

OakGPT Research | October 2026

Abstract

We instrumented local ROS Jazzy/Nav2/Gazebo command delivery while preserving independent C++ authorization, a 200 ms command lease and the 30 s navigation deadline. A reproduced 332.892270 ms mutex wait was localized to the Fast DDS participant send-resource mutex. Its original owner and equivalence to the historical 337 ms gap remain unproven. Middleware development comparisons contain startup, clock and TF failures and do not justify a corrected candidate. Phase 4 is NO-GO on the available evidence; held-out replication and final actual-motion safety regressions are not completed.

I. System and methods

The command route is Nav2 controller, ROS velocity guard, stamped DDS command, C++ ros_gz bridge, Gazebo transport, native expiry policy and simulated actuator. Passive traces record controller creation, guard callbacks, actual publication/take APIs, native ingress, PreUpdate and independent PostUpdate poses. Actual physics ECM contact pairs distinguish robot/environment contact from floor and self-contact. No physical stopping or real-world safety claim is made.

Cross-process differences use Linux CLOCK_BOOTTIME within one WSL kernel. DDS realtime, simulation time, Windows monotonic and Linux monotonic clocks are retained separately. Sequence/frame identifiers join guard publication to native decoding; no unique identity is available through the unstamped controller-to-guard segment. These results are API/observation delays, not wire timestamps or full controller-to-actuator latency bounds.

Current Fast DDS, explicit loopback UDP Fast DDS and Cyclone DDS are compared on the same two development worlds and three authorized instructions. Seeded order is reversed for the second world. Logging-on/off uses common compiled wrappers/contact configuration. Deep stack unwinding is disabled in the resumed passive matrix. Trials use fresh owned processes and do not resume faulted missions. Harness remediation and host contention preclude causal middleware efficacy claims.

II. Attribution and negative experiments

The captured waiter stack traverses pthread_mutex_lock, RTPSWriter::send_nts, LocatorSelectorSender, RTPSMessageGroup, StatefulWriter, DataWriter and rmw publication. Installed disassembly and Fast DDS 2.14.6 source identify m_send_resources_mutex_. The observed ordinary lock path is not bounded by the writer's 100 ms reliability max_blocking_time. Full guard API time was 350.829900 ms and includes diagnostic unwinding overhead. The 332.892270 ms wait itself precedes unwinding.

Later ownership tracking in another trial observes a flow-controller network-delivery thread holding the watched mutex for 63.165420 ms. This identifies a contender, not the owner of the original 332.9 ms wait. Transport-resource creation and cleanup are additional source-level possibilities without causal evidence for that event. CPU scheduling and network variability remain unresolved contributors.

Four matched SYNC/ASYNC development worlds completed 12/12 missions per arm. Maximum publication API time decreased from 321.779340 to 109.085850 ms, but maximum guard-begin/native delay was 322.156710 versus 329.688360 ms. ASYNC was not selected. A separate no-motor stopped-reader test published 300 zero commands with maximum API time 4.334760 ms; its reader was stopped for two seconds. That condition did not reproduce the long wait and is not a stopping test.

III. Availability and measured outcomes

backend-development-v1: 6/12 dispatched missions completed; 36 requests planned; 5/12 sessions unavailable.

backend-development-v2: 1/4 dispatched missions completed; 36 requests planned; 9/12 sessions unavailable.

Dispatched failure reasons across these cohorts: {"tf_watchdog": 7, "watchdog_not_ready_or_faulted": 2}. Unattempted requests are not successes or independent negative examples. Figure 1 includes startup-only traces; their timing cannot establish active mission reliability. Figure 2 separates unavailable sessions from dispatched failures.

The original clock harness expected plain text instead of C++ JSON, and a later parser dependency was unavailable in the ROS-only interpreter. A Windows shared-slot rename conflict was removed using immutable per-challenge files. These unsuccessful trials remain unchanged. Some corrected handshakes still exceeded one second and were rejected; no timeout was relaxed.

The mailbox uses fresh nonce binding, strict JSON/types/ranges, pre-existing challenge rejection and a one-second host-monotonic round-trip limit. It is trusted local transport, not an authenticated adversarial channel. Actual C++ integration accepted a fresh mission and rejected expired/future missions. The independent validator continues to read its own clock for mission acceptance.

IV. Verification and threats to validity

Executed software verification: 83 Python tests passed, 2 hardware tests skipped; Ruff lint/format and strict mypy passed. Release and ASan/UBSan C++ validator and expiry property tests passed. The 60-case C++ frozen oracle and actual C++ mailbox boundary check passed. ROS package tests: 10 passed, zero failures/errors/skips. Installed Gazebo expiry tests: 2 passed. Wheel/sdist packaging and 14 required native/ROS source checks passed. Phase 3.3 archive hashes verified (27 archives, 1958 members before the verification-log supplement).

Fresh native source snapshots and separate install/build directories prevent the stale-object problem found in earlier DrvFS builds. Source inputs and installed binary hashes are preserved. Actual

ECM contacts and native poses were observed. Earlier runs without working contact observation remain unavailable, not collision-free.

Trace writes use native tmpfs. Isolated microbenchmarks do not bound full application overhead. Deep stacks can extend a critical section and API duration; on/off file logging does not remove all interposition/formatting work. Terminal write loss may escape counters. Main-thread scheduler samples do not observe every middleware worker. Serial command samples are correlated, and only two reused development worlds are represented. No independent-cluster efficacy interval is justified.

Unrelated host activity was observed during the follow-up cohort: a per-process monotonic five-second sample showed approximately six CPU cores consumed by three unrelated applications. This is a measured confound, not proof of the mutex-delay cause. Missingness, protocol remediation and startup failures prevent a controlled ranking. Raw source/member hashes and failed runs are retained; model weights and rosbag data remain outside Git. No new GPU or LLM inference result is claimed.

V. Decision and next steps

NO-GO for Phase 4 on current evidence. No middleware candidate is justified by reliable development mission execution, so no candidate freeze or new 24-world held-out study has been performed. Final actual-motion fault regressions are also pending that gate. Prior-phase stopping evidence is historical and is not reclassified as a new pass.

The bounded mitigation remains fail-closed command expiration and independent mission authorization, without automatic uncertain-mission recovery. This bounds accepted command age, not physical stopping time or OS scheduling. The next action is a quiet-host controlled development comparison with the repaired harness and measured host load; only a justified candidate can proceed to the unchanged held-out and actual-motion safety protocol. Do not expand robotics scope or begin LoRA before this gate is resolved.

References

[1] eProsima, Fast DDS v2.14.6, RTPSParticipantImpl.h/.cpp and RTPSWriter.cpp, github.com/eProsima/Fast-DDS/tree/v2.14.6/src/cpp/rtps.

[2] ROS 2, rmw_fastrtps Jazzy, rmw_fastrtps_shared_cpp/src/rmw_publish.cpp, github.com/ros2/rmw_fastrtps/tree/jazzy.

[3] Gazebo Sim 8, systems/contact/Contact.cc, github.com/gazebosim/gz-sim/tree/gz-sim8/src/systems/contact.

[4] OakGPT, Phase 3.3 raw archive manifests, development summaries, docs/phase33-timing.md and verification-summary.json, repository eekeen/OakGPT.

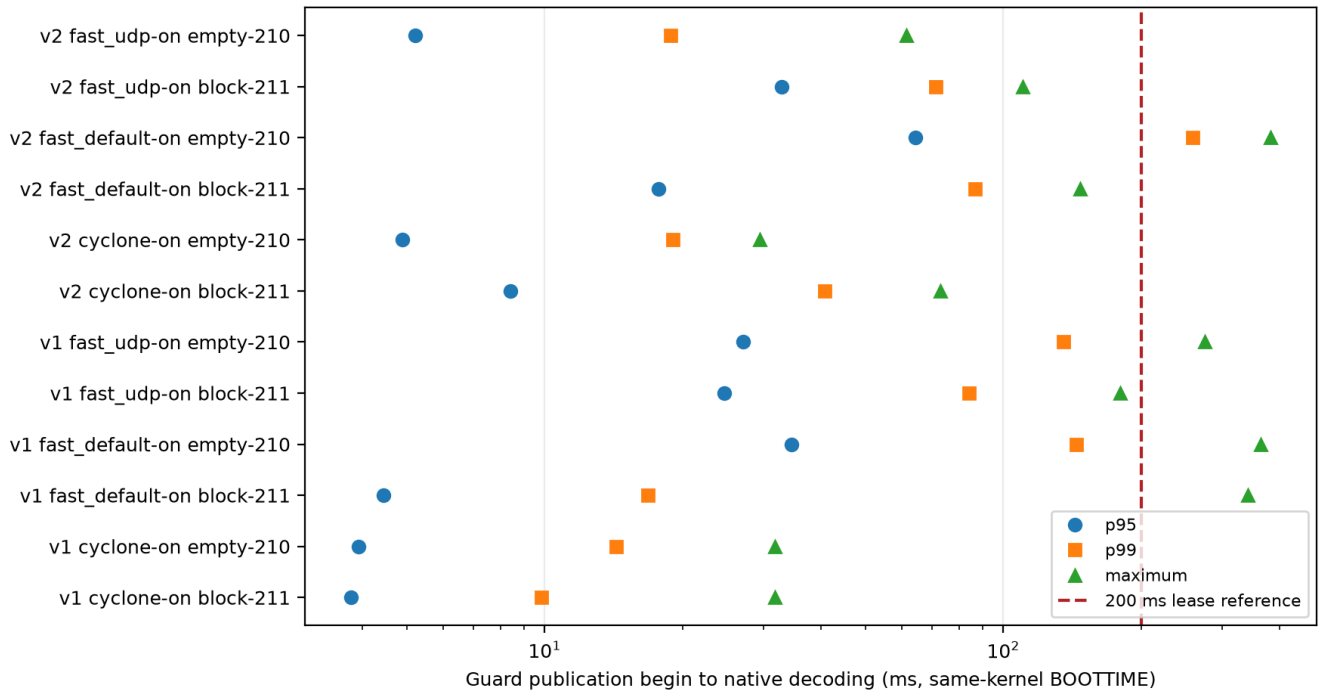


Fig. 1. Observed session tails, including startup-only sessions; logging-off sessions have no matched stage latency.

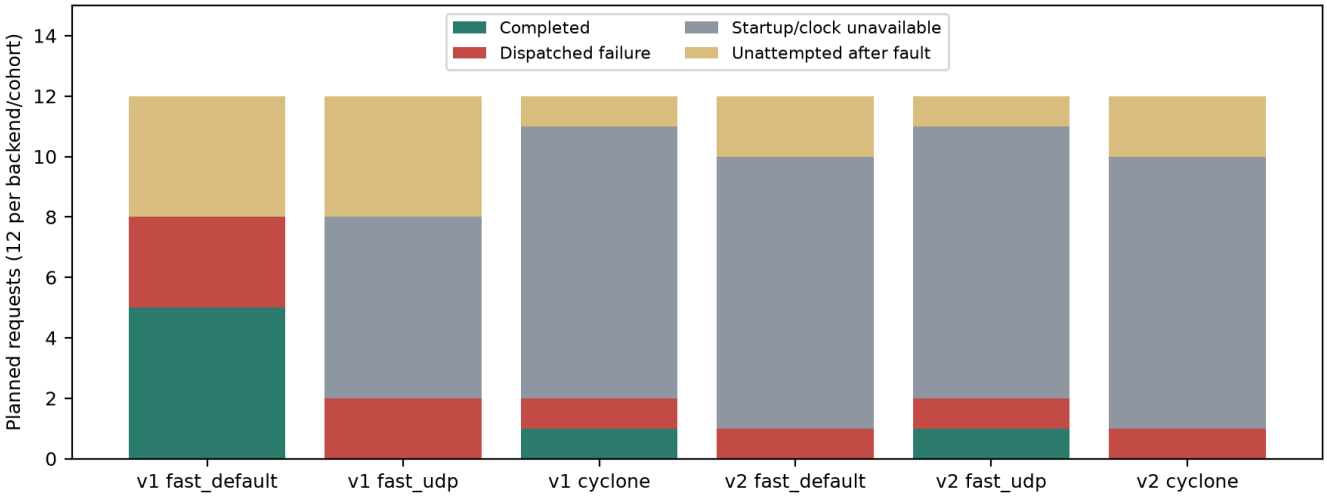


Fig. 2. All planned requests, preserving unavailable and unattempted outcomes. Development cohorts are confounded and do not establish middleware efficacy.